Instance Segmentation with Mask R-CNN and ConvNeXt Backbone

VRDL Homework 3

Hung-I Yang 112550097

May 8, 2026

1 Introduction

1.1 Task description

This homework is an *instance segmentation* task on colored medical images. The training/validation split contains 209 images, and the test set contains 101 images. Each image contains many cell instances belonging to four categories (`class1--class4`). Ground-truth masks are provided as `.tif` files where **each unique pixel value represents an instance**. The goal is to predict a set of instance masks for each test image and submit them in COCO-style RLE format. The competition metric is AP50.

1.2 Core idea

- **Model Architecture: Mask R-CNN + FPN. + ConvNeXt Backbone** I build a pipeline with Mask R-CNN [2] and Feature Pyramid Network (FPN) [3], which provides multi-scale features crucial for small, crowded instances.
- **Dense-scene adaptation (proposal and maxDets budget).** Many images contain hundreds of cells. The default `box_detections_per_img`=100 truncates predictions severely, hurting recall. Increasing detection/proposal budgets and aligning COCO evaluation `maxDets` with dense scenes improves both validation diagnostics and leaderboard performance.
- **Augmentation for small-data generalization.** With only 209 labeled images, the model overfits quickly. I use geometric and photometric augmentations (flip, random 90° rotation, color jitter), and additionally explore Copy-Paste augmentation [1] to reduce overfitting.

The training and inference code is available at `https://github.com/harry0816web/NYCU-VRDL/tree/main/hw3`.

2 Method

2.1 Data preprocessing and augmentation

The dataset provides a color `image.tif` and four class masks (`class1.tif--class4.tif`). Each class mask is an instance map where pixel values $\{0, 1, 2, \dots\}$ denote background and instance IDs. For each instance ID, I extract a binary mask, compute its bounding box, and store the sample in a Mask R-CNN compatible format. For competition submission, each predicted binary mask is encoded into COCO RLE.

I apply the following augmentations during training:

- **Geometric.** Random horizontal/vertical flips; random rotation by multiples of 90° (cells have no canonical orientation).
- **Photometric.** Color jitter (brightness/contrast/saturation), to simulate staining variation.
- **Copy-Paste (optional).** Randomly paste a small number of instances from another training image to increase diversity [1].
- **Multi-scale resizing.** Resize such that the shorter edge is sampled from {640, 672, 704, 736, 768, 800} with max size 1024.

Validation uses deterministic resizing (short edge 800, max size 1024) without stochastic augmentation.

2.2 Model architecture

2.2.1 Mask R-CNN with FPN

I adopt Mask R-CNN [2] as the core instance segmentation framework. It consists of (i) an RPN that proposes candidate regions; (ii) a RoI box head that classifies and regresses bounding boxes; and (iii) a RoI mask head that predicts per-instance masks. To support objects at different scales, an FPN [3] aggregates multi-resolution features from the backbone.

2.2.2 Backbone (ConvNeXt-Small [4])

Relative to a classic ResNet-style backbone, ConvNeXt [4] modernizes the convolutional stack with ideas borrowed from hierarchical vision Transformers: a patchify stem (non-overlapping 4×4 convolution with stride 4 instead of a large 7×7 conv plus max-pooling), depthwise 7×7 convolutions for a wider effective receptive field, inverted bottleneck blocks, LayerNorm in place of BatchNorm, GELU activations, and dedicated 2×2 stride-2 downsampling stages between levels.

I use **ConvNeXt-Small** pretrained on ImageNet-1K. Four hierarchical stage outputs (strides 4/8/16/32 with channel widths $[96, 192, 384, 768]$) feed the FPN with 256-channel lateral and output convolutions plus a max-pooled top level. The full detector (backbone + FPN + Mask R-CNN heads) totals ∼70M parameters, well below the 200M cap.

The first two backbone stages are frozen so that low-level filters stay near their ImageNet solution; a spatial dropout layer (p=0.1) is applied to backbone feature maps before the FPN during training; and weight decay is set to 10^{-3}. These regularization choices are discussed in Section 3.1.5.

2.2.3 Dense-scene inference settings

To better handle crowded scenes, I increase key inference-time budgets in Mask R-CNN (RPN proposals and per-image detections). Additionally, I use a slightly higher box NMS threshold (0.7) to reduce over-suppression when nearby objects overlap.

2.3 Hyperparameters and training strategy

Training uses the standard Mask R-CNN multi-task loss (RPN objectness/regression, RoI classification/regression, and mask BCE). Due to the small dataset size, I use a small batch size with gradient accumulation. Table 1 summarizes the final configuration (0.5555 on the public leaderboard).

Table 1: Final training configuration (Mask R-CNN + ConvNeXt-Small + FPN).

Field	Value
Backbone / Neck / Head	ConvNeXt-Small [4] / FPN [3] / Mask R-CNN [2]
Total parameters	$\sim 7\times10^7$ (<200M)
Backbone freeze / FPN dropout	`freeze_stages` = 2 / `Dropout2d` p=0.1 (pre-FPN)
Optimizer	SGD (lr = 0.005, momentum = 0.9)
`weight_decay`	0.001
`batch_size` / grad accumulation	1 / 2
`epochs`	48 (warmup: 3 epochs)
Multi-scale `min_sizes` / `max_size`	(640,672,704,736,768,800) / 1024
`box_detections_per_img`	1000
`box_nms_thresh`	0.7
RPN top-N (train pre/post)	4000 / 3000
RPN top-N (test pre/post)	3000 / 2000
LR scheduler	StepLR (`step_size`=20, `gamma`=0.1)

3 Results

3.1 Training & Validation Curves

3.1.1 Backbone comparison: ResNet vs. ConvNeXt

To isolate the effect of the backbone, I compare a ResNet baseline against a ConvNeXt backbone under the same Mask R-CNN [2] + FPN [3] framework. ConvNeXt [4] modernizes the classic ResNet-style design (e.g., patchify stem, large-kernel depthwise conv, inverted bottleneck, LayerNorm/GELU), which empirically yields stronger features for dense small-instance segmentation.

Since this phase is purely a backbone performance comparison, I did not record the baseline validation loss (only training loss and validation AP for diagnosis). Figure 1 shows the validation AP curves, where ConvNeXt achieves substantially stronger validation performance than ResNet; Table 2 further confirms the same trend on the test set.

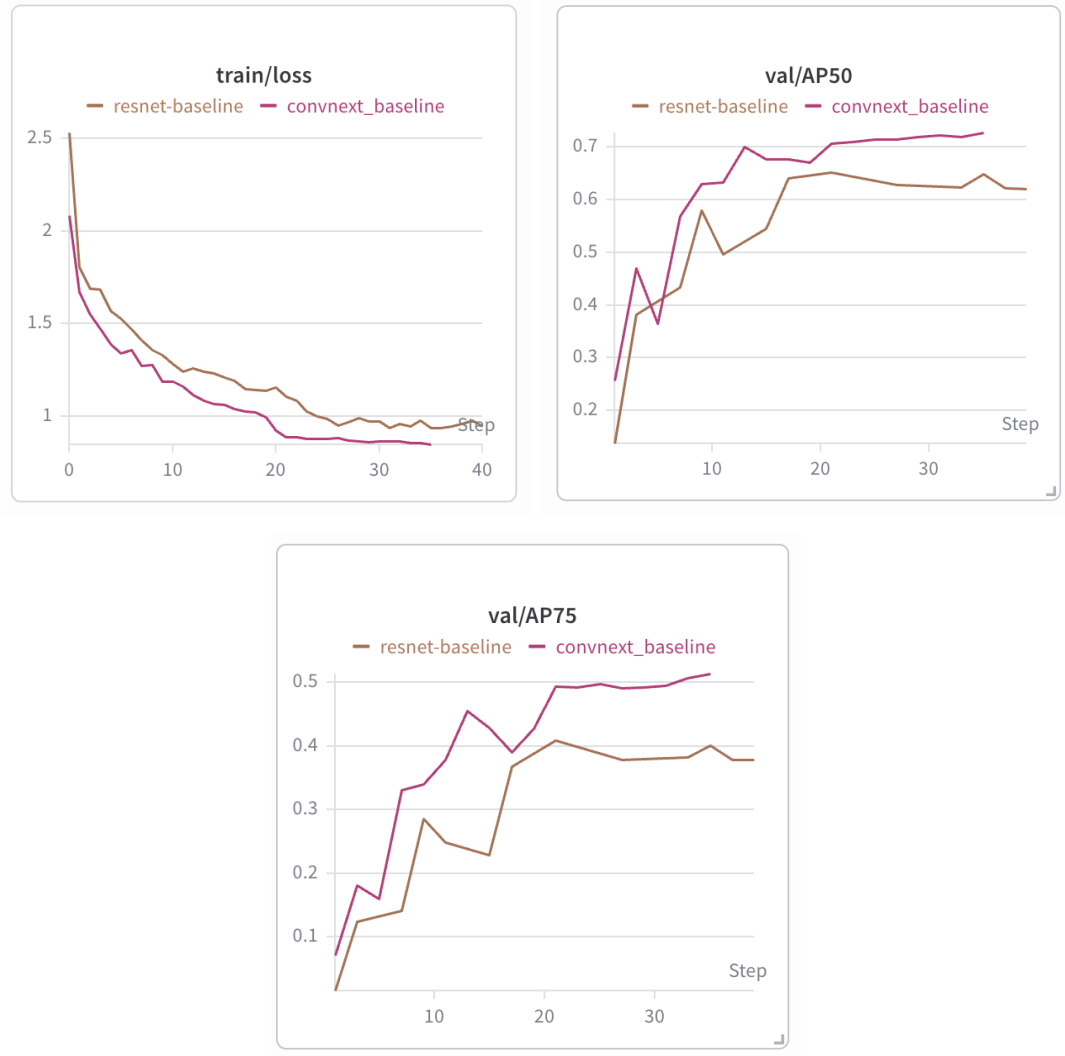


Figure 1: Backbone comparison setup: training loss (top); validation AP@50 and AP@75 (bottom).

Table 2: Test AP50: ResNet vs. ConvNeXt backbone.

Setting	AP50
ResNet baseline	0.38
ConvNeXt baseline	0.47

3.1.2 Full-image inference and stronger augmentation

To prevent OOM under the dense prediction budget, the baseline initially relied on tiled inference at test time, which introduced three issues: (i) a *scale mismatch* because tiles were fed at a different resolution than the training resize pipeline expects; (ii) *boundary truncation* that splits cells across tiles, producing low-confidence partial detections; and (iii) *brittle merge logic* (Soft-NMS) that cannot distinguish duplicate detections from genuinely adjacent cells in dense scenes. After upgrading to an RTX 5090 (32 GB VRAM), full-image inference became feasible and removed all three failure modes at once.

Given the limited training set size (∼178 images), I strengthen data augmentation by adding color jitter and random 90° rotations (Section 2.1) to simulate staining variation and the lack of

canonical cell orientation, together with a linear warmup for three epochs and a longer training run of 48 epochs. Note that this stage does not include a baseline validation-loss curve either; the key comparison is on AP.

Figure 2 shows that validation AP is clearly higher than the baseline, and Table 3 further confirms a substantial test-score improvement (0.47 → 0.52).

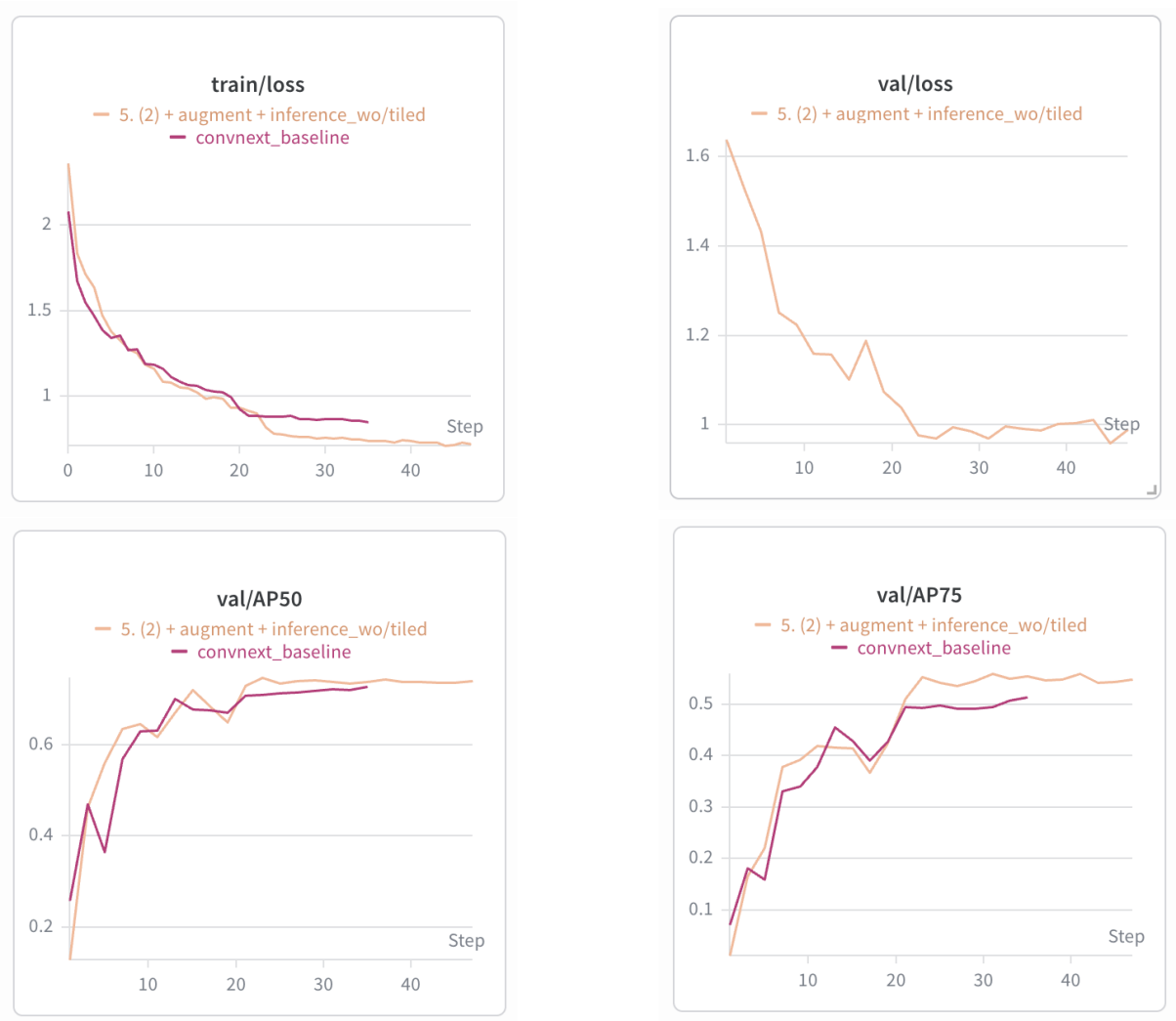


Figure 2: Full-image inference with augmentation and warmup: losses (top); validation AP@50 and AP@75 (bottom).

Table 3: Test AP50: effect of full-image inference and stronger augmentation.

Setting	AP50
Baseline (tiled inference)	0.47
+ augmentation + warmup + full-image inference	0.52

3.1.3 Higher-resolution training (failed experiment)

A natural next step was to raise resolution so that the model sees finer cell boundaries. I increased multi-scale short edges to 800–960 with `max_size`=1333. However, this change did not help: validation AP did not improve and the test score regressed from 0.52 to 0.51. In addition,

the validation loss curve typically stops decreasing around epoch 20, indicating an overfitting issue under this higher-resolution setting that would require additional regularization.

Figure 3 and Table 4 summarize this ablation.

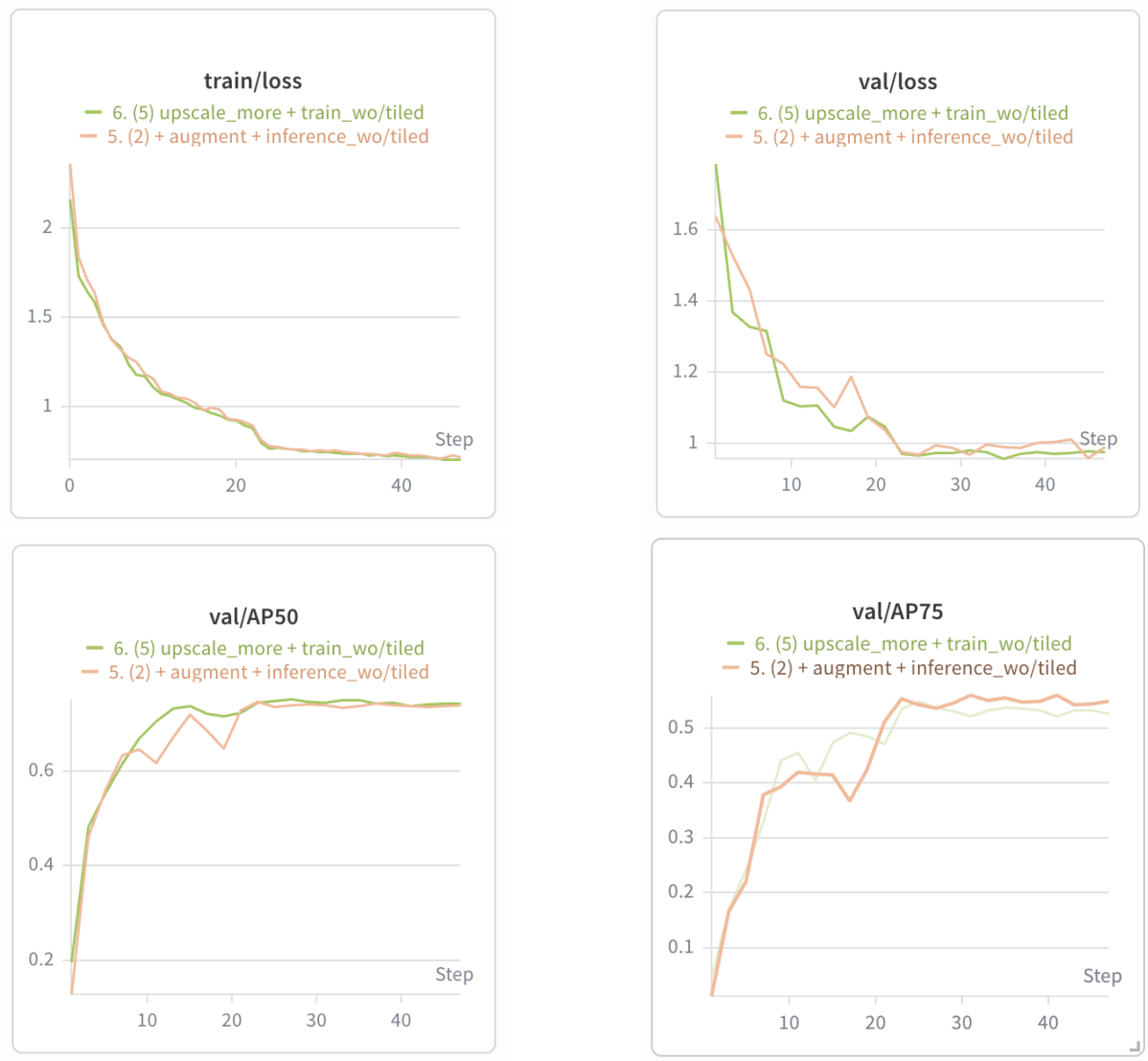


Figure 3: High-resolution experiment: losses (top); validation AP@50 and AP@75 (bottom).

Table 4: Test AP50: effect of higher-resolution training (reverted).

Setting	AP50
Full-image + augmentation (640–800 / 1024)	0.52
+ upscale (`min_sizes`=800–960, `max_size`=1333)	0.51

3.1.4 Copy-Paste augmentation

Since higher resolution failed to improve generalization, I turned to data-level regularization instead. Following Simple Copy-Paste [1], training samples a donor image with 50% probability, pastes 1–30 instances onto the current image, removes originals occluded >70%, and merges surviving boxes/masks. This creates combinatorially many unique instance layouts ($178^2 \approx 31$k image pairs) with realistic cell textures, acting as a strong layout-level regularizer for the small

training set. Although the validation curves in Figure 4 do not show a clear improvement, the test AP50 still increases from 0.52 to 0.54 (Table 5).

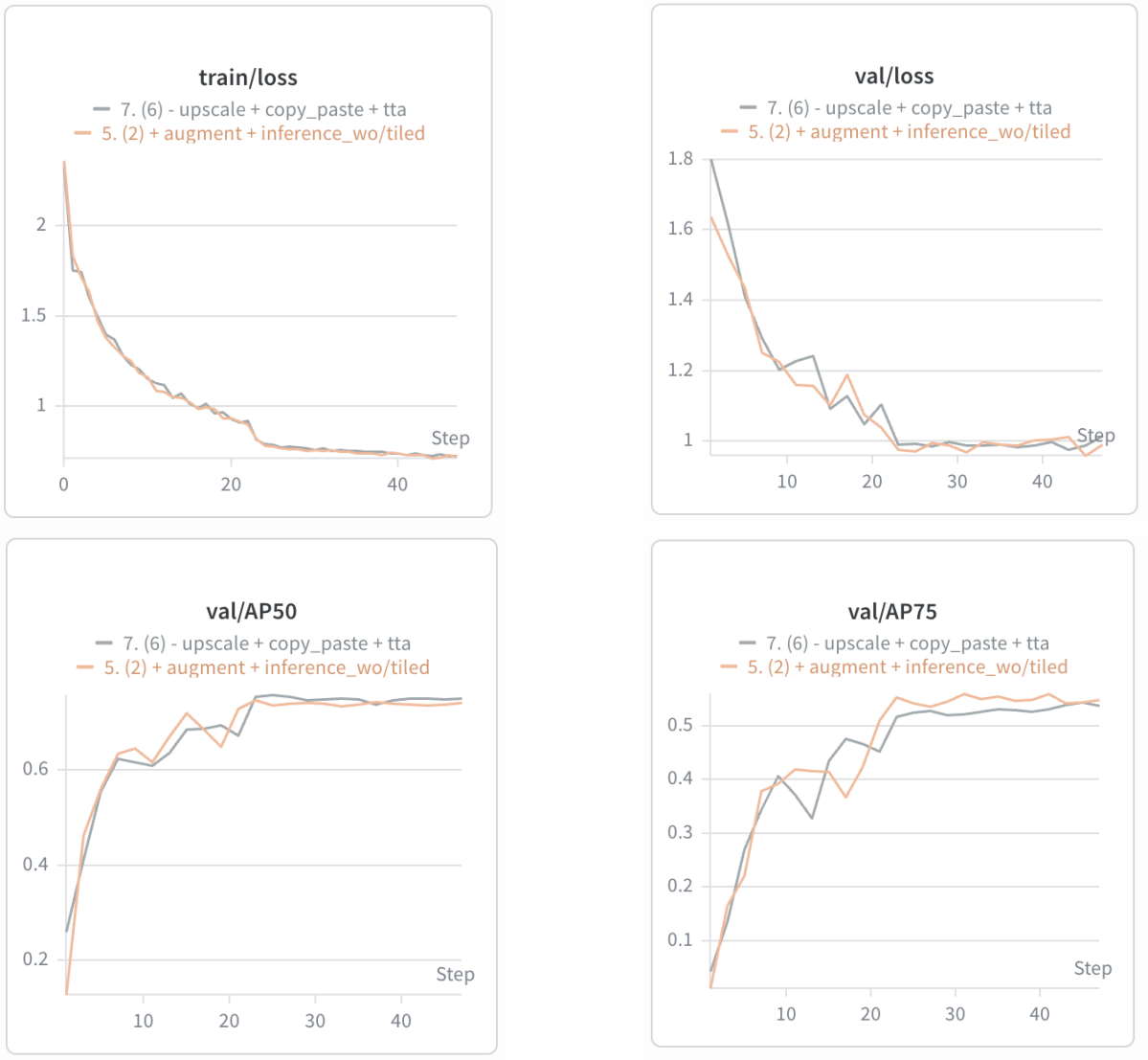


Figure 4: Copy-Paste augmentation: losses (top); validation AP@50 and AP@75 (bottom).

Table 5: Test AP50: Copy-Paste augmentation.

Setting	AP50
Previous best model	0.52
+ Copy-Paste augmentation	0.54

3.1.5 ConvNeXt-Small backbone and anti-overfitting regularization

Although Copy-Paste improved AP50 by +0.02, the marginal gain from data-level augmentation appeared to be plateauing: more layout diversity no longer translated into proportional metric improvement. This motivated a backbone upgrade. I replace ConvNeXt-Tiny (∼28M backbone parameters) with ConvNeXt-Small, which deepens stage 2 from 9 to 27 blocks while keeping the same channel widths, raising total parameters to ∼70M (still <200M).

However, the larger backbone overfits more severely on only ∼178 images: validation loss climbs noticeably after epoch 20. To counteract this without sacrificing the newly gained capacity, I apply three regularization measures: (i) freeze the first two backbone stages so that low-level features stay near their well-trained ImageNet solution; (ii) insert spatial dropout (p=0.1) between the backbone and FPN to prevent over-reliance on specific feature channels; and (iii) increase weight decay from 5×10^{-4} to 10^{-3}.

Figure 5 shows the resulting training dynamics; Table 6 reports the stepwise test numbers.

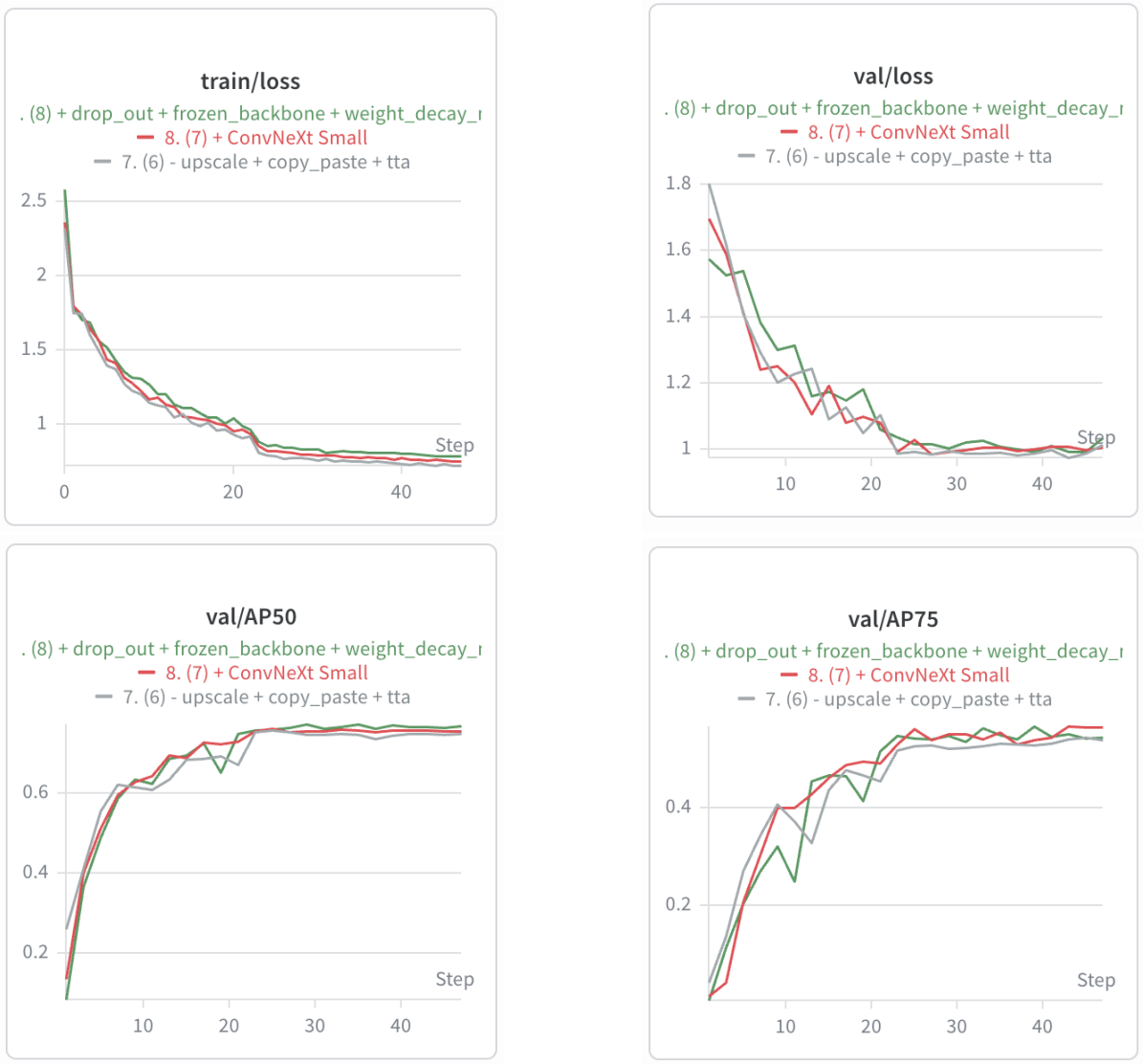


Figure 5: ConvNeXt-Small with regularization: losses (top); validation AP@50 and AP@75 (bottom).

Table 6: Test AP50: backbone upgrade and anti-overfitting regularization.

Setting	AP50
Previous best model	0.54
+ ConvNeXt-Small	0.5457
+ freeze first two stages + FPN dropout + higher weight decay	**0.5555**

Table 7: Progressive test AP50 summary (public leaderboard).

Stage	Main change	AP50
(1)	Baseline: ConvNeXt-Tiny, `maxDets`=1000, multi-scale, dense tiling	0.47
(2)	Full-image inference, color jitter + 90° rot., warmup	0.52
(3)	Higher res. 800–960 / 1333 (failed)	0.51
(4)	+ Copy-Paste [1]	0.54
(5a)	+ ConvNeXt-Small backbone	0.5457
(5b)	+ freeze low stages + FPN dropout + weight decay ↑	**0.5555**

4 Additional Experiments

4.1 TTA with mask IoU fusion

Hypothesis. A single forward pass may miss borderline instances or produce noisy confidence scores. By running inference under multiple geometric views that mirror training augmentations and fusing the results, consistently detected instances should receive reinforced scores while single-view false positives are down-weighted. Because dense cell images contain many spatially adjacent instances, the fusion mechanism must avoid score suppression based on spatial overlap; instead, we use pixel-level mask IoU to match predictions across views.

How this may (or may not) work. The pipeline consists of two phases.

Phase 1 — Generating six views. Each view runs a full Mask R-CNN forward pass independently; all predictions are mapped back to the original image coordinate system before fusion.

1. **Original@800.** Standard inference at short-edge 800; serves as the *base set*.
2. **Horizontal flip / Vertical flip.** The image is flipped, inference runs, then bounding-box coordinates and masks are inverse-flipped.
3. **90° / 270° rotation.** The image is first padded to a square (capped at max side 1024 to prevent OOM; images exceeding this are resized before padding). After rotated inference, bounding-box corners are inverse-rotated, masks are rotated back with `np.rot90`, and the padding region is cropped. If the image was resized, coordinates are scaled back to the original resolution.
4. **Scale-640.** The model's internal resize target is temporarily changed to short-edge 640; the framework automatically maps predictions back to the original image size.

Phase 2 — Mask IoU fusion. With the base set (original@800) as anchor, each augmented view is fused in turn:

1. **Per-class grouping.** Only predictions of the same class are compared.
2. **Mask IoU matrix.** For each class, a pairwise mask IoU matrix between augmented and base predictions is computed efficiently via RLE-encoded masks.
3. **Greedy matching.** In each round the highest-IoU pair is selected; if IoU ≥ 0.3, the augmented score is accumulated into the matched base prediction and both entries are invalidated for subsequent rounds.
4. **Score averaging.** After all five augmented views are processed, each base prediction's final score is divided by the total number of views (6), so that instances consistently detected across views retain a high score while single-view false positives are naturally down-weighted.

5. **Recovering missed instances.** Unmatched augmented predictions with score ≥ 0.7 are checked against the current fused set; those with mask IoU < 0.3 to every existing instance *of the same class* are added as new detections (with their score also divided by 6), recovering instances that the base view missed.

The key design principle is that **scores are never suppressed based on overlap with other detections**—only averaged across views. However, the averaging step inherently dilutes the score of any detection that is not consistently found across all views: a base detection matched by k out of 5 augmented views receives a final score of $(s_{\text{base}} + s_1 + \cdots + s_k)/6$, which can be substantially lower than s_{base} when k is small.

Result and analysis. Table 8 shows that TTA with mask IoU fusion slightly *decreases* AP50 from 0.5555 to 0.5534. Two likely explanations:

Table 8: Test AP50: effect of TTA with mask IoU fusion.

Setting	AP50
Best single-model (ConvNeXt-Small + regularization)	**0.5555**
+ TTA mask IoU fusion (6 views)	0.5534

1. **Score dilution on small objects.** Over 90% of instances are COCO-small. The scale-640 view reduces spatial resolution and is likely to miss many of these small cells; rotation views with zero-padded borders can similarly fail near image edges. Each such "empty vote" divides a correct detection's score by 6 without contributing a match, pushing borderline true positives below the detection threshold and reducing recall.

2. **Mask misalignment from geometric transforms.** Inverse-rotating and cropping masks introduces sub-pixel misalignment for small, irregular cell shapes. When this pushes mask IoU below the 0.3 matching threshold, the same instance is treated as unmatched in both the base and augmented sets, diluting its averaged score while simultaneously injecting a near-duplicate through the recovery step.

References

[1] Golnaz Ghiasi, Yin Cui, Aravind Srinivas, Ruoming Qian, Tsung-Yi Lin, Ekin D. Cubuk, Quoc V. Le, and Barret Zoph. Simple copy-paste is a strong data augmentation method for instance segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021.

[2] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask R-CNN. In *IEEE International Conference on Computer Vision (ICCV)*, 2017.

[3] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan, and Serge Belongie. Feature pyramid networks for object detection. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017.

[4] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.